Step 1 — One Specialist's Drafting Logic, Working Alone, No Agent Behavior — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 1: a team is only as good as each member. Start with one specialist — the Writer, because it needs no tools — and prove its core skill alone: findings in, article out, as one LCEL chain. **If not:** in Step 5, a bad article could come from the Writer, the Reviewer, the routing or the state, and you'd be debugging all four at once.

After this step the project runs: `python -m contentforge.main` writes three test articles from made-up findings.

The shared files

The same files as Projects 1-3, with the package name changed to `contentforge`: Doc01's config, errors and logger, Project 1's prompt loader, and Project 3's `get_model()`.

Story — `exceptions.py`, `config.py`, `utils/logger.py`, `utils/prompts.py`: settings load and fail loudly at startup, logging is set up once, prompts live in `.txt` files. **If not:** each agent would repeat this setup its own way.

```
# src/contentforge/exceptions.py
# why: one file for every named error in the project, so any module can
# raise or catch a specific problem instead of a bare Exception.

# from Doc01
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

```
# src/contentforge/config.py
# how: os.getenv() is how we read env vars
import os
# why: @dataclass auto-builds __init__ for a typed holder
from dataclasses import dataclass
# how: reads .env and loads it into os.environ
from dotenv import load_dotenv
```

```
# why: a specific error type, not a bare Exception
from contentforge.exceptions import MissingConfigError

@dataclass
class Config:
    # why: a typed object, not a plain dict — your editor autocompletes
    # .openai_api_key and .log_level, and catches a typo'd field name.
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    # why: one place that turns a missing/empty env var into a clear,
    # named error, instead of repeating the same if-check for every key.
    # when: called once per required key, inside load_config().
    value = os.getenv(key)
    if value is None or value == "":
        # how: both "never set" (None) and "set but blank" (empty string)
        # count as missing — an empty API key is never usable either way.
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    # why: the one function every other file calls to get settings — so a
    # missing key fails loudly here, at startup, not confusingly later.
    # when: call this once, at the top of main(), before anything else.
    # how: reads .env into os.environ; must run before os.getenv() below
    load_dotenv()
    # how: required — raises if missing
    api_key = require_env("OPENAI_API_KEY")
    # how: optional — "INFO" if unset
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

```
# src/contentforge/utils/logger.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    # why: gives every file in the project the same logging setup from one
    # place, instead of each file configuring handlers its own way.
    # when: logging.getLogger(name) always returns the same object for the
    # same name, so calling this again in the same run is safe and cheap.
    logger = logging.getLogger(name)
```

```
    if not logger.handlers:
        # how: only attach a handler the first time this name is
        # configured — without this guard, every line would print twice.
        level_name = os.getenv("LOG_LEVEL", "INFO")
        # how: falls back to INFO if the env value is invalid
        level = getattr(logging, level_name.upper(), logging.INFO)

        # how: prints to the console
        handler = logging.StreamHandler()
        handler.setLevel(level)
        # why: every line carries a timestamp/source/level, not just the
        # bare message
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        formatter = logging.Formatter(log_format)
        handler.setFormatter(formatter)

        logger.addHandler(handler)
        # how: the logger's own level must be at least this low, or the
        # handler never even sees the message
        logger.setLevel(level)

    return logger
```

```
# src/contentforge/utils/prompts.py

# how: a path from the repo root — every command in this project is run
# from the repo root, the same rule the tests and scripts follow
PROMPTS_DIR = "src/contentforge/prompts"

def load_prompt(name: str) -> str:
    # why: prompts live in .txt files, not inside Python strings — you
    # can reword an agent's behavior without touching its code.
    # when: called whenever an agent needs its system prompt.
    # how: e.g. "worker" -> "src/contentforge/prompts/worker.txt"
    path = PROMPTS_DIR + "/" + name + ".txt"
    with open(path) as prompt_file:
        # how: strip() drops the blank line at the end of the file
        return prompt_file.read().strip()
```

Also create empty `__init__.py` files in `utils/`, `models/` and `agents/`.

Back to this step · Hint 1 · Hint 2 · Solution

Solution — a reusable Writer with strict rules, tested on 3 inputs

Story — `models/llm.py` **:** one place builds the model for every agent — the same `get_model()` as Project 3, since Step 5's graph nodes only receive the state. **If not:** four agents would build four models with four settings.

```
# src/contentforge/models/llm.py
from langchain_openai import ChatOpenAI

# why: one place to change the model for every agent
MODEL_NAME = "gpt-4o-mini"

def get_model() -> ChatOpenAI:
    # why: graph nodes only receive the state, so each node builds its
    # model here instead of being handed one
    # how: ChatOpenAI reads OPENAI_API_KEY from the environment, which
    # load_config() filled from .env (and checked) at startup
    return ChatOpenAI(model=MODEL_NAME, temperature=0)
```

Story — `prompts/writer.txt` **:** the Writer's rules: a titled 150-250-word article, **only** the given findings, and keep each fact's `[source]` tag. **If not:** the article could quietly mix in facts nobody researched.

`src/contentforge/prompts/writer.txt`

```
You are the Writer on a content team. Write a short article (150-250
words) with a title, using ONLY the findings below. Do not add facts
that are not in the findings. Keep each fact's [source] tag.

Findings:
{findings}
```

(Step 2 adds a feedback section to this prompt.)

Story — `agents/writer_agent.py` **:** `build_writer_chain()` is the Writer's whole skill; `run_writer()` is the one function other code calls. **If not:** the chain would live inside `main.py`, and Step 2 couldn't wrap it.

```
# src/contentforge/agents/writer_agent.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from contentforge.models.llm import get_model
from contentforge.utils.prompts import load_prompt

def build_writer_chain():
    # why: the Writer's whole skill is one LCEL chain (Doc05) — proven
    # alone here, before any other agent exists
    prompt = ChatPromptTemplate.from_template(load_prompt("writer"))
    return prompt | get_model() | StrOutputParser()

def run_writer(findings: str) -> str:
    chain = build_writer_chain()
    return chain.invoke({"findings": findings})
```

Story — `main.py`: three test inputs, including a nearly empty one **on purpose** — to see now what the Writer does with almost nothing, before Research ever feeds it something thin. **If not:** you'd first see that behavior in the full pipeline, with four agents to suspect.

```
# src/contentforge/main.py
import sys

from contentforge.agents.writer_agent import run_writer
from contentforge.config import load_config
from contentforge.exceptions import MissingConfigError

# why: made-up findings for now — Research and Analysis don't exist yet.
# The last one is nearly empty ON PURPOSE, to see what the chain does
# with almost nothing to work from.
TEST_FINDINGS = [
    "Range:\n- A typical e-bike battery gives 40 to 100 km per charge. "
    "[battery-review]\n- Most e-bikes are pedal-assist. [bike-basics]",
    "Savings:\n- Remote workers save 72 minutes a day of commuting. "
    "[commute-study]\n- Loneliness is the main downside. [wellbeing]",
    "- Solar panels exist.",
]

def main() -> None:
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)
```

```
    for findings in TEST_FINDINGS:
        print("=" * 60)
        print(run_writer(findings))

if __name__ == "__main__":
    main()
```

Run it: `python -m contentforge.main`, and **read** all three articles. For the thin input, a good Writer writes something very short, or says there's too little to go on — it shouldn't invent facts about solar panels. If it invents, tighten the "ONLY the findings" wording now.